Trabalho prático: um protocolo de transporte confiável sobre UDP

Redes de Computadores — UFC, Campus Russas

Prazo: 3 semanas

Em uma frase

Você vai escrever as duas pontas de um protocolo de transporte: um **emissor** que quebra um texto grande em pacotes e um **receptor** que remonta o texto *exatamente igual*, mesmo que a rede perca, atrase ou embaralhe pacotes e confirmações.

Ao final, você terá implementado, em escala reduzida, as ideias centrais do TCP e do QUIC: números de sequência, confirmações seletivas (SACK), janela de envio, detecção de perdas por confirmações posteriores ("retransmissão rápida") e por temporizador (timeout).

Sumário

1 O problema **2**

2 Conceitos que você vai usar **2**
- 2.1 Números de sequência e intervalos . . . 2
- 2.2 Confirmações com intervalos (SACK) . . . 2
- 2.3 Bytes em voo e janela . . . 3
- 2.4 Como o emissor descobre que um pacote se perdeu . . . 3

3 Arquitetura do código-base **4**

4 O que você deve implementar **4**
- 4.1 `Receiver` . . . 4
- 4.2 `Sender` . . . 5

5 Ambiente e execução passo a passo **5**
- 5.1 Pré-requisitos . . . 5
- 5.2 Estrutura dos arquivos . . . 5
- 5.3 Criando o ambiente virtual . . . 6
- 5.4 Executando o programa de verdade (UDP) . . . 6
- 5.5 Executando os testes . . . 7
- 5.6 Calculando a nota estimada . . . 7
- 5.7 Usando o simulador diretamente . . . 7
- 5.8 Gerando as visualizações . . . 8
- 5.9 Ciclo de trabalho recomendado . . . 8
- 5.10 Problemas comuns de ambiente . . . 9

6 Fluxos que o seu protocolo precisa tratar **9**

7 Testes e avaliação **10**

8 Plano sugerido para as três semanas 11

9 Dicas de depuração e erros comuns 11

10 Regras e entrega 12

A Para saber mais 12

1 O problema

O emissor recebe uma mensagem arbitrariamente grande e precisa enviá-la ao receptor em pacotes de no máximo 1500 bytes (até `payload_size` = 1200 bytes de dados mais a formatação JSON). No caminho, pacotes podem ser **perdidos**, **atrasados** ou chegar **fora de ordem**. O receptor remonta a mensagem e, para *cada* pacote recebido, devolve uma confirmação (ACK) dizendo tudo o que já recebeu. Com essas confirmações, o emissor *deduz* o que se perdeu e retransmite.

Os pacotes viajam sobre **UDP**, que não oferece nenhuma garantia: ele apenas acrescenta ao IP as portas de origem e destino (e um checksum), para que o sistema operacional saiba a qual programa entregar cada datagrama. Toda a confiabilidade é responsabilidade do *seu* código.

Objetivos de aprendizagem. Ao terminar o trabalho você deve ser capaz de:

1. explicar por que números de sequência por *byte* e confirmações por *intervalo* permitem remontar uma mensagem sob perdas e reordenação;
2. manter, no receptor, um buffer que entrega dados à aplicação *em ordem e sem repetição*;
3. controlar, no emissor, quantos bytes estão "em voo" e respeitar uma janela;
4. distinguir os dois mecanismos de detecção de perda (ACKs posteriores e timeout) e saber quando cada um é necessário;
5. raciocinar sobre casos de borda: ACKs perdidos, retransmissões perdidas, retransmissões desnecessárias (espúrias) e duplicatas.

2 Conceitos que você vai usar

2.1 Números de sequência e intervalos

Cada *byte* da mensagem tem um número de sequência: o primeiro é 0, o segundo é 1, e assim por diante.[1] Um pacote de dados carrega um **intervalo semiaberto** [início, fim): o pacote `(1200, 2400)` contém os bytes 1200 a 2399. O tamanho do pacote é fim − início.

Pacotes de dados contêm: o número de sequência do primeiro byte, o do último byte +1, os dados e um **identificador único** (`id`). Usamos JSON para serializar, o que simplifica a depuração (protocolos reais usam cabeçalhos binários compactos, como o cabeçalho TCP):

```
{"type": "data", "seq": [1200, 2400], "id": 5, "payload": "...1200 caracteres..."}
```

2.2 Confirmações com intervalos (SACK)

O ACK leva o `id` do pacote que o provocou e a **lista de todos os intervalos já recebidos** (desde o byte 0), ordenada e com intervalos encostados unidos:

```
{"type": "ack", "sacks": [[0, 600], [700, 800]], "id": 9}
```

[1] O TCP usa números de 32 bits, que "dão a volta" depois de ∼4 GB e começam em um valor aleatório por segurança. Aqui ignoramos isso: os números começam em 0 e podem crescer sem limite.

Exemplo

Se chegaram os pacotes 0–100, 100–200, 500–600, 300–400 e 700–800, o receptor responde `[0-200, 300-400, 500-600, 700-800]`. Se em seguida chegam 200–300 e 400–500, a lista vira apenas `[0-600, 700-800]`. Sem perdas nem reordenação, um único intervalo basta.

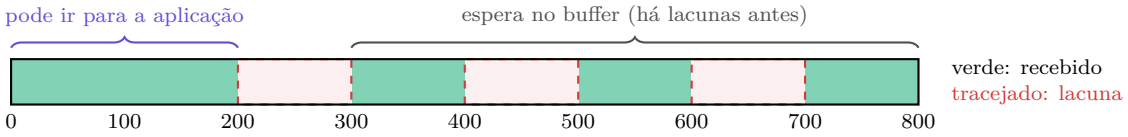


Figura 1: Estado do receptor no primeiro momento do exemplo. Só o prefixo contíguo (0–200) pode ser entregue à aplicação; o resto aguarda as lacunas serem preenchidas.

Curiosidade. O cabeçalho TCP só tem espaço para 3 ou 4 intervalos de SACK (bits eram caros quando a Internet foi projetada). O **QUIC**, protocolo que também roda sobre UDP e hoje transporta boa parte do tráfego web, permite listas arbitrárias de intervalos — como o nosso. O QUIC também numera cada *pacote* com um identificador único, nunca reutilizado nem em retransmissões; adotamos a mesma ideia com o campo `id`.

2.3 Bytes em voo e janela

Mandar pacotes demais de uma vez é arriscado: a rede pode descartá-los e o receptor pode não ter memória para guardá-los. Por isso o emissor limita os bytes **em voo**:

$$\text{em voo} = \text{enviados} - \text{confirmados} - \text{presumidos perdidos}.$$

O código-base guarda esse valor na variável `inflight` e só pede um novo pacote ao seu `Sender` quando `inflight + packet_size < recv_window`. Neste trabalho a janela (`--recv_window`, padrão 15000 bytes) é constante. Em protocolos reais ela varia: o receptor anuncia quanto espaço livre tem no buffer (*controle de fluxo*) e um algoritmo de *controle de congestionamento* estima quanto a rede aguenta — assunto de outro trabalho.

2.4 Como o emissor descobre que um pacote se perdeu

Não existe "ACK negativo". A perda é sempre uma *suposição*, feita de dois jeitos:

Por ACKs posteriores (retransmissão rápida). Se chegam confirmações de vários pacotes enviados *depois* do pacote P, e P continua sem confirmação, P provavelmente se perdeu. O TCP usa a regra dos "3 ACKs duplicados"; com `id`s únicos, podemos comparar diretamente "quem foi enviado antes de quem". É rápido: não espera nenhum temporizador.

Por timeout. Se o emissor fica 0,5 s sem receber *nenhum* ACK, o código-base zera `inflight` e chama `Sender.timeout()`. É o último recurso: funciona sempre, mas é lento. Ele é *indispensável* quando não há pacotes posteriores para gerar ACKs — por exemplo, quando o último pacote se perde.

Pense antes de codificar

Por que um *único* ACK posterior não é evidência suficiente de perda? O que acontece com a sua regra quando a rede *reordena* pacotes? E quando o pacote "perdido" só estava atrasado e chega depois da retransmissão?

3 Arquitetura do código-base

O arquivo `src/transport.py` já contém toda a parte de rede. Você completa duas classes. A Figura 2 mostra quem chama quem.

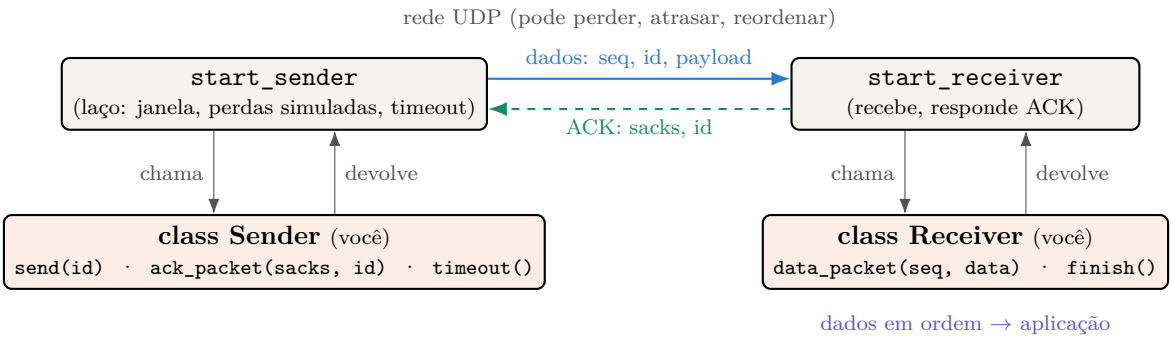


Figura 2: O código-base cuida dos sockets; suas classes decidem *o que* enviar e *o que* confirmar.

O laço do emissor (`start_sender`), em palavras. Leia o código junto com esta descrição:

1. Se há espaço na janela, pede um intervalo a `sender.send(packet_id)`.
   - `None`: a transmissão acabou; envia `fin` e espera a confirmação `fin`.
   - intervalo vazio `(x, x)`: não há nada a enviar agora; passa a esperar ACKs.
   - caso contrário: soma o tamanho a `inflight`, "sorteia" se o pacote se perde (`--simloss`), envia (ou retém para embaralhar, com `--pkts_to_reorder`) e incrementa `packet_id` — *mesmo se o pacote foi perdido.*
2. Se não há espaço (ou não havia o que enviar), espera um ACK por até 0,5 s.
   - chegou ACK: com probabilidade `simloss` ele é descartado; senão faz `inflight -= sender.ack_packet(sacks, id)` e verifica `assert inflight >= 0`;
   - passou 0,5 s: `inflight = 0` e `sender.timeout()`.

Não altere `start_sender`, `start_receiver` nem `main`

Os testes executam essas funções exatamente como estão. Você pode criar quantas funções auxiliares, atributos e métodos quiser, desde que dentro de `transport.py`.

4 O que você deve implementar

4.1 `Receiver`

`data_packet(seq_range, data)`
chamado a cada pacote de dados. Devolve `(sacks, dados_para_aplicacao)`:

- `sacks`: lista de *todos* os intervalos recebidos até agora, ordenada, sem sobreposição, com intervalos encostados unidos (ex.: `[(0, 600), (700, 800)]`);
- `dados_para_aplicacao`: os bytes que *agora* podem ser entregues, isto é, os que continuam o que já foi entregue, sem lacunas. Pode ser `"`.

`finish()`
chamado quando chega o `fin`. Opcional; útil para verificar se sobrou algo no buffer.

Invariante do receptor: a concatenação de tudo o que foi entregue à aplicação é *sempre* um prefixo da mensagem original. Nenhum byte é entregue duas vezes nem fora de ordem.

4.2 `Sender`

`send(packet_id)`
devolve o intervalo a enviar agora (até `payload_size` bytes e *sem bytes já confirmados*); `(x, x)` se não há nada a enviar no momento; `None` somente quando tudo foi enviado *e confirmado*.

`ack_packet(sacks, packet_id)`
processa um ACK e devolve quantos bytes *deixaram de estar em voo* por causa dele — confirmados ou presumidos perdidos. O valor é **incremental**: cada byte enviado é descontado no máximo uma vez.

`timeout()`
chamado após 0,5 s sem ACK; nesse momento `start_sender` já zerou `inflight`. Seu estado deve ficar coerente com isso.

O invariante de `inflight`

Para cada pacote, `start_sender` *soma* seus bytes a `inflight` ao enviá-lo. Esses bytes devem ser *subtraídos exatamente uma vez* pelo retorno de `ack_packet` — ou deixam de contar porque houve um timeout. Contar duas vezes dispara `assert inflight >= 0`; nunca contar faz a janela "encolher" até o emissor parar. Os casos traiçoeiros: ACK atrasado que chega depois de um timeout; pacote dado como perdido cujo ACK chega depois; ACK perdido cujos bytes aparecem no SACK do ACK seguinte.

5 Ambiente e execução passo a passo

Esta seção mostra, do zero, como preparar o ambiente e executar *tudo* o que acompanha o trabalho: o programa, os testes, a nota estimada, o simulador e as visualizações. Os comandos assumem um terminal Linux, macOS ou WSL; as diferenças para o Windows nativo estão indicadas.

5.1 Pré-requisitos

- **Python 3.9 ou mais recente.** Para conferir, rode `python3 --version`; no Windows, `py --version`.
- **Sistema:** Linux, macOS ou Windows com WSL (recomendado). No Windows nativo tudo funciona, mas os testes não têm limite de tempo individual (um laço infinito trava a suíte até você apertar `Ctrl+C`).
- **Bibliotecas:** `pytest` (testes) e `matplotlib` (figuras). O pacote `lorem` é opcional. Todas estão em `requirements.txt`.

5.2 Estrutura dos arquivos

Descompacte `trabalho-transporte-aluno.zip`. Você terá:

```
trabalho-transporte/
  requirements.txt          dependências
  enunciado/                este documento
  src/
    transport.py            <- o ÚNICO arquivo que você edita e entrega
    generate_bogus_text.py  gera um texto de teste
  testes/
    test_1_receptor.py ...  suíte usada na correção (7 arquivos)
    netsim.py               simulador determinístico
    corrigir.py             calcula a nota estimada
  visualizacao/
    visualizar.py           desenha a execução de um cenário
```

```
    cenarios.py             definição dos cenários
    figuras-referencia/     como os cenários ficam com uma solução correta
```

5.3 Criando o ambiente virtual

Um ambiente virtual isola as bibliotecas do trabalho das do sistema. Faça isto **uma vez**, na pasta `trabalho-transporte`:

```
cd trabalho-transporte
python3 -m venv .venv                # cria o ambiente na pasta .venv
source .venv/bin/activate            # ativa (Linux/macOS/WSL)
#   Windows (PowerShell):  .venv\Scripts\Activate.ps1
#   Windows (cmd):         .venv\Scripts\activate.bat
python -m pip install -r requirements.txt
```

Com o ambiente ativo, o prompt mostra `(.venv)` e o comando `python` já aponta para o Python certo. **Toda vez que abrir um terminal novo**, ative de novo com `source .venv/bin/activate`. Para conferir a instalação:

```
python -c "import pytest, matplotlib; print('ok')"
```

5.4 Executando o programa de verdade (UDP)

1. Gere um arquivo para enviar. Qualquer texto ASCII serve; o gerador produz o tamanho pedido (aproximadamente):

```
cd src
python generate_bogus_text.py 1000000 > test_file.txt
```

2. Inicie o receptor em um terminal (ele fica rodando até você apertar `Ctrl+C`):

```
python transport.py --ip localhost --port 7000 receiver
```

3. Inicie o emissor em *outro* terminal (com o ambiente ativado), na pasta `src`:

```
python transport.py --ip localhost --port 7000 --sendfile test_file.txt sender
```

O emissor imprime cada pacote enviado e cada ACK recebido e termina com `Got FIN-ACK`. O receptor imprime `received data (summary):` seguido do início do texto e do número de bytes recebidos — confira se é igual ao tamanho do arquivo (`wc -c test_file.txt`). Como as mensagens são muitas, você pode descartá-las e ver só o final: `... sender | tail -5`.

4. Varie as condições da rede acrescentando opções ao **emissor**:

Opção	Efeito
`--simloss 0.1`	descarta 10% dos pacotes de dados *e* dos ACKs (teste 0, 0.01, 0.1, 0.5, 0.9)
`--pkts_to_reorder 5`	embaralha grupos de 5 pacotes consecutivos (1 = sem reordenação)
`--recv_window 3000`	muda a janela em bytes (padrão 15000; 3000 = um pacote em voo)

Exemplo combinando tudo, com um arquivo menor para terminar rápido:

```
python generate_bogus_text.py 100000 > pequeno.txt
python transport.py --ip localhost --port 7000 --sendfile pequeno.txt \
       --simloss 0.2 --pkts_to_reorder 5 sender
```

Opcional: Mahimahi. O emulador Mahimahi (só Linux) cria perdas "de verdade": rode `mm-loss uplink 0.1`, inicie o receptor *fora* do shell criado, com `--ip 0.0.0.0`, e o emissor *dentro* dele usando o IP em `$MAHIMAHI_BASE_IP` (normalmente `100.64.0.1`). Não é necessário para este trabalho.

5.5 Executando os testes

Todos os comandos abaixo são executados **dentro da pasta `testes`**. Por padrão os testes usam `../src/transport.py`, ou seja, a sua implementação.

```
cd testes
python -m pytest -q                          # todos os testes (resumo)
python -m pytest -q test_1_receptor.py       # uma categoria
python -m pytest -q -x                       # para no primeiro teste que falhar
python -m pytest -v test_2_emissor.py        # lista cada teste com PASSED/FAILED
python -m pytest -q -k sobreposicao          # só testes cujo nome contém "sobreposicao"
python -m pytest -q "test_1_receptor.py::test_exemplo_do_enunciado"   # um teste
```

Como ler uma falha. O pytest mostra o arquivo e a linha do teste, o `assert` que falhou e uma mensagem em português explicando o que era esperado, por exemplo:

```
AssertionError: bytes depois de uma lacuna NÃO podem ser entregues à aplicação
```

Abra o teste (os arquivos são curtos e comentados): ele mostra exatamente a sequência de chamadas que provocou o erro, e você pode reproduzi-la sozinho. Nos testes de simulação, a mensagem começa com `ERRO:` quando o seu código lançou exceção (o traceback aparece logo abaixo) ou descreve o que deu errado na transferência (dados diferentes, não terminou, `inflight` negativo...).

Testar outro arquivo (por exemplo, uma cópia de backup):

```
TRANSPORT_IMPL=../src/transport_v2.py python -m pytest -q     # Linux/macOS/WSL
# Windows (cmd):
set TRANSPORT_IMPL=..\src\transport_v2.py
python -m pytest -q
```

5.6 Calculando a nota estimada

```
cd testes
python corrigir.py                           # usa ../src/transport.py
python corrigir.py --semente 7               # muda as sementes aleatórias
python corrigir.py --saida relatorio.md      # também salva o relatório em arquivo
python corrigir.py --impl outro/transport.py # testa outro arquivo
```

O relatório mostra, para cada categoria, quantos testes passaram e os pontos obtidos, a nota final e a lista dos testes que falharam com o motivo. A suíte completa leva poucos segundos com uma implementação correta; se demorar muito, provavelmente algum teste está esbarrando no limite de tempo.

5.7 Usando o simulador diretamente

O arquivo `netsim.py` pode ser usado em scripts seus para investigar um caso específico, com perdas exatamente onde você quiser. Crie, por exemplo, `testes/meu_cenario.py`:

```
from netsim import Simulation, load_impl, random_text, drop_ids, drop_ack_ids

impl = load_impl("../src/transport.py")
dados = random_text(20000, seed=1)
res = Simulation(impl, dados, recv_window=7500,
                 drop_data=drop_ids({3, 4})).run()   # perde os pacotes de id 3 e 4
print(res.summary())       # pacotes, timeouts, bytes retransmitidos...
print(res.explain())       # "ok" ou o motivo da falha
for ev in res.events[:20]: # eventos: send, arrive, ack, timeout, fin
    print(ev)
```

e execute com `python meu_cenario.py` dentro da pasta `testes`. Parâmetros de `Simulation`:

Parâmetro	Significado
`recv_window`	janela em bytes (padrão 15000)
`loss`, `ack_loss`	probabilidade de perder dados / ACKs (`ack_loss` padrão = `loss`)
`pkts_to_reorder`	igual à opção do programa: embaralha grupos de k pacotes
`seed`	semente das perdas aleatórias (mesma semente ⇒ mesma execução)
`drop_data=drop_ids({...})`	perde os pacotes de dados com esses ids
`drop_ack=drop_ack_ids({...})`	perde os ACKs gerados por esses ids
`extra_delay={id: atraso}`	atrasa um pacote específico (provoca reordenação)

5.8 Gerando as visualizações

Os comandos são executados **dentro da pasta `visualizacao`** e, por padrão, usam a sua implementação (`../src/transport.py`):

```
cd visualizacao
python visualizar.py --lista                        # nomes dos cenários
python visualizar.py perda-de-ack                   # um cenário
python visualizar.py todos                          # todos os cenários
python visualizar.py perda-de-ack --sem-texto       # só a figura, sem o traço
python visualizar.py todos --saida minhas_figuras   # outra pasta de saída
python visualizar.py perda-de-ack --impl ../src/transport_v2.py
```

Cada execução imprime no terminal o **traço** do cenário (um evento por linha: envio, chegada, ACK, timeout, com o valor devolvido por cada método seu) e salva a figura em `figuras/<cenario>.png`. Compare com `figuras-referencia/<cenario>.png`: diferenças na quantidade de retransmissões ou de timeouts apontam onde investigar.

Para criar um cenário seu, copie uma das entradas do dicionário `CENARIOS` em `cenarios.py`, dê outro nome e ajuste os campos `tamanho`, `recv_window`, `drop_data`, `drop_ack`, `extra_delay` ou `pkts_to_reorder`. O novo cenário aparece automaticamente em `--lista`.

5.9 Ciclo de trabalho recomendado

1. Edite `src/transport.py`.
2. Rode os testes da categoria em que está trabalhando (por exemplo, `python -m pytest -q -x test_1_receptor.py`).
3. Se um teste falhar e a mensagem não bastar, reproduza o caso com o simulador ou com a visualização correspondente e leia o traço.
4. Quando a categoria passar, rode a suíte inteira (para não quebrar o que já funcionava) e `python corrigir.py`.
5. De vez em quando, rode o programa real com UDP e perdas (`--simloss`).

5.10 Problemas comuns de ambiente

Sintoma	Causa e solução
`No module named pytest` (ou `matplotlib`)	ambiente virtual não ativado; rode `source .venv/bin/activate`
`python: command not found`	use `python3` ou ative o ambiente virtual (dentro dele, `python` existe)
`Address already in use`	outro receptor ainda está rodando, ou a porta não foi liberada; feche-o ou use outra porta (acima de 1024)
`ConnectionRefusedError` no emissor	o receptor não está rodando, ou está em outra porta/IP
emissor termina com `Timeout` e código 1	o `fin` não foi confirmado: o receptor não recebeu nenhum dado (ex.: arquivo vazio ou receptor reiniciado no meio)
testes ficam "parados"	laço infinito no seu código; no Linux/macOS o teste é interrompido após 20 s; no Windows, `Ctrl+C`
`ERRO: ... lançou ...` nos testes	exceção no seu código; o traceback mostra a linha

6 Fluxos que o seu protocolo precisa tratar

As figuras desta seção foram geradas pelo script `visualizacao/visualizar.py` a partir de um simulador determinístico (Seções 5 e 7). Em cada figura: à esquerda, o diagrama de sequência (tempo cresce para baixo; setas azuis são dados, laranja são retransmissões, verdes tracejadas são ACKs, X é perda); à direita, o gráfico *tempo* × *número de sequência* e os bytes em voo. **Rode o mesmo script com a sua implementação** e compare: é a melhor ferramenta de depuração deste trabalho.

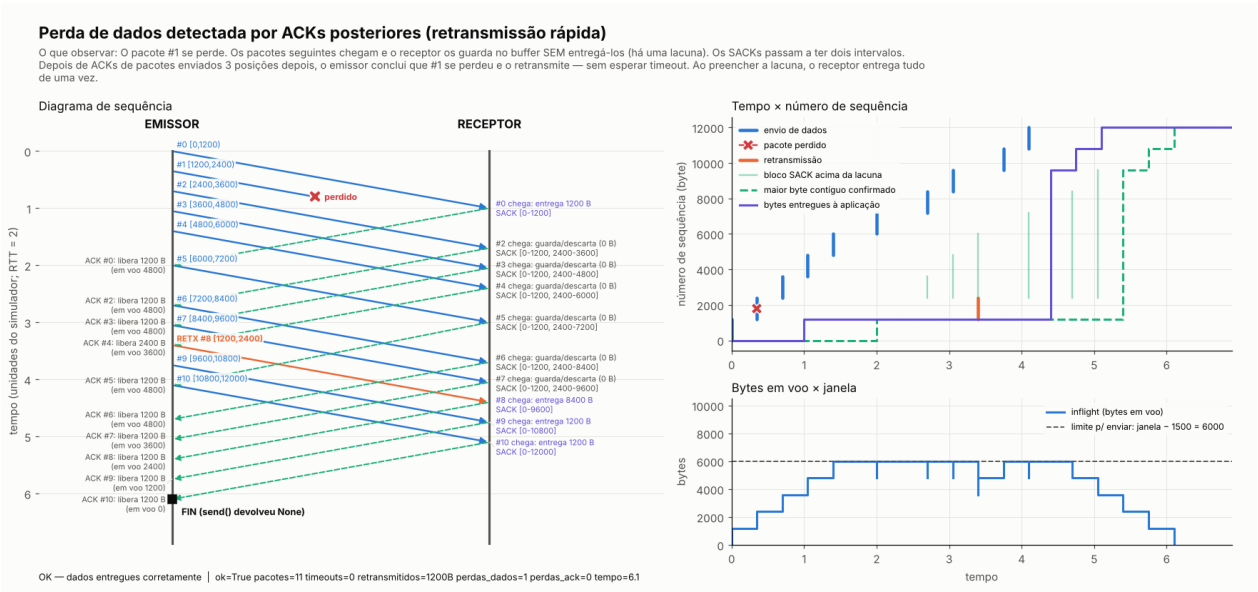


Figura 3: Perda do pacote #1 recuperada por ACKs posteriores. Note os SACKs com dois intervalos, o receptor guardando dados sem entregá-los e a entrega "em bloco" quando a lacuna é preenchida.

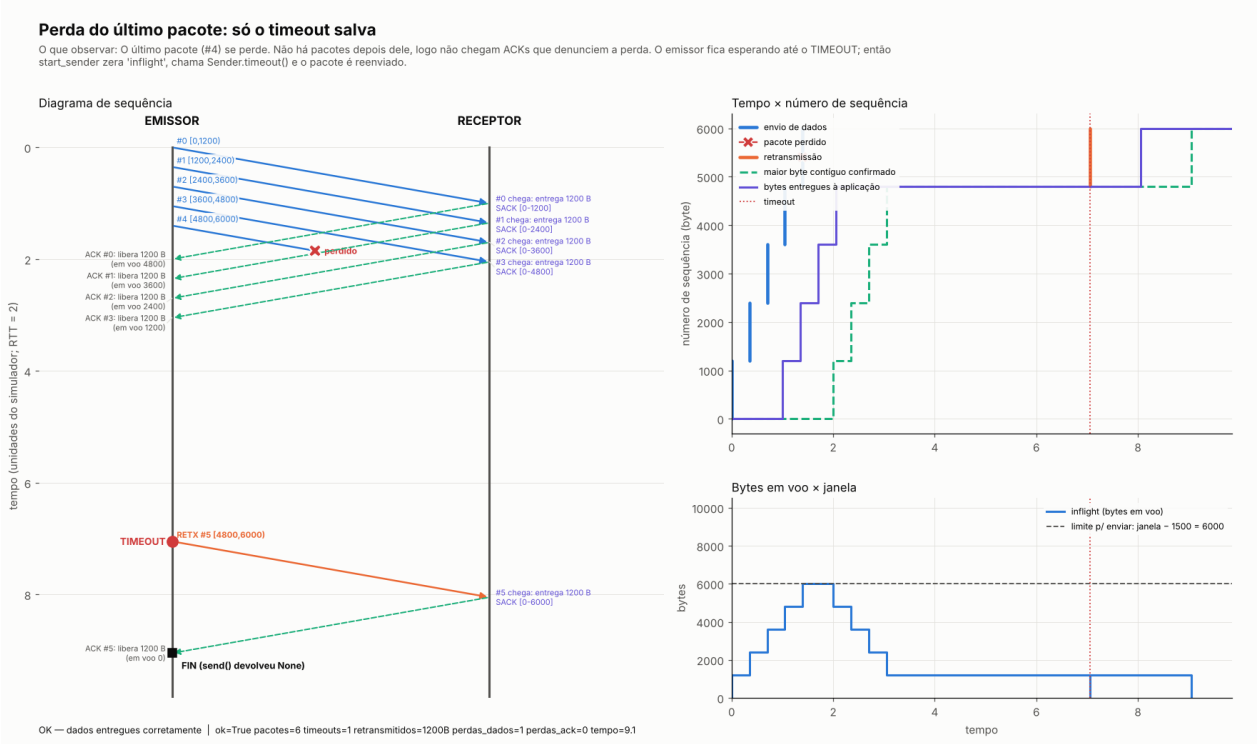


Figura 4: Perda do último pacote: sem pacotes posteriores, nada denuncia a perda. Só o timeout resolve.

A Tabela 1 lista todos os cenários disponíveis; como gerá-los está na Seção 5.

Cenário	O que ele mostra
`caminho-feliz`	janela deslizante sem perdas; cada ACK abre espaço para um novo pacote
`perda-retransmissao-rapida`	perda detectada por ACKs posteriores, sem timeout (Fig. 3)
`perda-de-ack`	ACK perdido; o SACK seguinte cobre o pacote e nada é retransmitido
`perda-na-cauda-timeout`	último pacote perdido; só o timeout recupera (Fig. 4)
`retransmissao-perdida`	a própria retransmissão se perde; a detecção recomeça
`rajada-de-perdas`	vários pacotes seguidos perdidos; SACK evita reenviar o que chegou
`reordenacao`	pacote atrasado chega depois dos seguintes; buffer no receptor
`reordenacao-retransmissao-espuria`	atraso grande gera retransmissão desnecessária e duplicata
`janela-pare-e-espere`	janela de 1 pacote: toda perda custa um timeout
`buffer-de-reordenacao-do-simulador`	efeito de `--pkts_to_reorder`: pacotes retidos e timeouts no fim

Tabela 1: Cenários de visualização.

7 Testes e avaliação

Você recebe a mesma suíte de testes usada na correção (pasta `testes/`). Ela tem três camadas:

1. **testes unitários**, que chamam diretamente os métodos de `Receiver` e `Sender`;
2. **simulador determinístico** (arquivo `netsim.py` na pasta `testes`), que reproduz exatamente o laço de `start_sender` e `start_receiver` em tempo virtual, com perdas e atrasos controlados, sem sockets — rápido e reprodutível;
3. **testes com UDP real**, que executam o seu `transport.py` como programa.

Os comandos para executá-los estão na Seção 5.

Categoria (arquivo)	O que verifica	Peso
receptor (`test_1`)	buffer, entrega em ordem, duplicatas, sobreposições, formato dos SACKs	20
emissor (`test_2`)	envio contíguo, `payload_size`, intervalo vazio vs. `None`, retorno incremental	15
perdas (`test_3`)	retransmissão rápida, timeout, ACK atrasado, retransmissão perdida, rajadas	15
integração (`test_4`)	transferências completas com 0–90% de perda, janelas pequena e grande	20
reordenação (`test_5`)	`pkts_to_reorder` de 2 a 10, atrasos, retransmissões espúrias	15
eficiência (`test_6`)	nenhuma retransmissão sem perdas, uso da janela, recuperação sem timeout	5
UDP (`test_7`)	o programa real, com e sem perdas e reordenação	10
Total		**100**

Tabela 2: Categorias de testes. Dentro de cada categoria, todos os testes valem o mesmo.

A pontuação $P \in [0, 100]$ vira nota por

$$\text{nota} = 1 + 9 \cdot \frac{P}{100},$$

de modo que a nota vai de 1,0 a 10,0. Observações:

- A reordenação, que era bônus na versão original deste trabalho, agora faz parte da nota (15 pontos).
- Na correção, as sementes aleatórias serão trocadas (`python corrigir.py --semente N`). Uma implementação correta passa com qualquer semente; "decorar" casos não funciona.
- Cada teste tem um limite de tempo (20 s; 120 s nos de UDP). Laço infinito conta como falha.

8 Plano sugerido para as três semanas

Semana 1 — intervalos e receptor. Leia todo o código-base. Escreva e teste as funções auxiliares de intervalos. Implemente o `Receiver` até passar em `test_1_receptor.py`. Desenhe à mão o exemplo da Figura 1.

Semana 2 — emissor sem perdas, depois timeout. Implemente `send` e `ack_packet` para o caso sem perdas (`test_2_emissor.py`). Em seguida, `timeout()`: só com ele o protocolo já é *confiável* (lento, mas correto). Rode `test_4_integracao.py`.

Semana 3 — retransmissão rápida, reordenação e polimento. Detecte perdas por ACKs posteriores (`test_3_perdas.py`, `test_6_eficiencia.py`), trate reordenação e retransmissões espúrias (`test_5_reordenacao.py`), e rode o programa real com UDP.

9 Dicas de depuração e erros comuns

- **Comece pelo que é verificável isoladamente.** Uma função de "unir intervalos" errada quebra receptor e emissor ao mesmo tempo. Teste-a com casos de borda: intervalos que se encostam, contidos, fora de ordem, vazios.
- **Use o simulador em vez dos sockets** enquanto desenvolve: é determinístico e mostra a causa do erro. Os testes imprimem mensagens explicando o que era esperado.
- **Use os cenários de visualização** com a sua implementação e compare com o texto "o que observar". O traço textual no terminal mostra o retorno de cada chamada.
- **Atributos de classe vs. de instância**: `buffer = {}` no corpo da classe é compartilhado por todos os objetos. Use `self.buffer` em `__init__`.

- **Os SACKs chegam ao emissor como listas** (`[[0, 1200]]`), não tuplas, porque passaram por JSON.
- **`None` cedo demais**: terminar quando tudo foi *enviado* (e não *confirmado*) perde dados.
- **Retransmitir bytes já confirmados** é proibido e desperdiça a rede; confira os SACKs mais recentes antes de reenviar.
- **`AssertionError: inflight >= 0`** quase sempre é dupla contagem; releia o quadro "O invariante de `inflight`".
- **Emissor parado até o timeout** sem motivo aparente: algum byte nunca saiu de "em voo", ou `send` devolveu `(x, x)` quando havia algo a retransmitir.

10 Regras e entrega

- Entregue **apenas** o arquivo `transport.py`, com as classes `Receiver` e `Sender` completas.
- Não altere `start_sender`, `start_receiver` nem `main`.
- Use apenas a biblioteca padrão do Python. Não use bibliotecas que já implementem conjuntos de intervalos, árvores de intervalos ou protocolos de transporte.
- É permitido usar estruturas de dados e algoritmos simples e ineficientes, se deixarem o código mais claro (dentro do razoável: os testes têm limite de tempo).
- Discutir ideias com colegas é bem-vindo; compartilhar código, não.

A Para saber mais

- Peterson & Davie, *Computer Networks: A Systems Approach*, seção 5.2.4 (janela deslizante e controle de fluxo) e capítulo 6 (controle de congestionamento).
- Estrutura do cabeçalho TCP: https://en.wikipedia.org/wiki/Transmission_Control_Protocol.
- QUIC (RFC 9000) e sua detecção de perdas (RFC 9002), que inspira o uso de `id`s únicos por pacote.
- Encerramento de conexão no TCP (*connection termination*): por que o nosso `fin` é simplificado e o que pode dar errado se ele se perder.